

مشروع نظام مشاركة الملفات والملاحظات بين الطلاب باستخدام تقنية P2P

عبدالله برهم

طلال السيد

علي أحمد

أحمد موسى

مقدمة:

يهدف هذا المشروع إلى إنشاء نظام بسيط وعملي لمشاركة الملفات والملاحظات بين الطلاب باستخدام مفهوم Peer-to-Peer (P2P) أي الاتصال المباشر بين الأجهزة دون الحاجة إلى خادم مركزي رئيسي.

تم تطوير المشروع باستخدام لغة البرمجة Python بالاعتماد على مكتبات الشبكات والاتصالات، بحيث يستطيع كل مستخدم أن يعمل كخادم (Server) وعميل (Client) في الوقت نفسه.

الفكرة الأساسية للمشروع هي تسهيل تبادل الملفات الدراسية والملاحظات والرسائل بين الطلاب ضمن نفس الشبكة المحلية، مع توفير بيئة تواصل مباشرة وسريعة وآمنة نسبياً، دون الاعتماد على خدمات الإنترنت الخارجية أو التخزين السحابي.

يعتبر هذا المشروع مثلاً عملياً على مفاهيم الشبكات، والبرمجة متعددة الخيوط (Multithreading)، والاتصالات عبر بروتوكول TCP/IP، بالإضافة إلى تطبيقات أنظمة الند للند (P2P Systems) المستخدمة في العديد من الأنظمة الحديثة.

1. فكرة المشروع

وتعتمد على إنشاء تطبيق يعمل على أجهزة متعددة داخل نفس الشبكة، بحيث يساعد الطلاب على:

- 1- اكتشاف الأجهزة الأخرى المتصلة بالشبكة.
- 2- إرسال واستقبال الملفات.
- 3- إنشاء ومشاركة الملاحظات الدراسية.
- 4- تبادل الرسائل النصية.
- 5- إجراء بث جماعي للرسائل.

حيث في الأنظمة التقليدية تعتمد عملية نقل الملفات على خادم مركزي يقوم بتخزين البيانات وإدارتها، أما في هذا المشروع فكل جهاز يمثل "عقدة" (Node) مستقلة تستطيع الإرسال والاستقبال مباشرة.

وبالتالي عندما يريد أحد الطلاب مشاركة ملف لا يتم رفعه إلى خادم مركزي بل يتم إرساله مباشرة من جهاز الطالب المرسل إلى جهاز الطالب المستقبل. مما يقلل من الاعتماد على الإنترنت ويزيد سرعة نقل البيانات داخل الشبكة المحلية.

2. أهداف المشروع

- ◀ تطبيق مفهوم Peer-to-Peer عملياً، وفهم كيفية عمل الأنظمة اللامركزية في الشبكات.
- ◀ تطوير نظام مشاركة ملفات
- ◀ إنشاء نظام يسمح للطلاب بمشاركة الملفات الدراسية منها:
 - ملفات PDF
 - العروض التقديمية
 - الصور
 - الأكواد البرمجية
 - المستندات النصية
- ◀ إنشاء نظام ملاحظات
- ◀ تطبيق مفاهيم الشبكات
- ◀ المساعدة في فهم:
 - بروتوكول TCP
 - عناوين IP

- المنافذ Ports
- الاتصال بين الأجهزة
- إرسال واستقبال البيانات

◀ تعلم البرمجة متعددة الخيوط

- حيث نستخدم تقنية الـ Threads للسماح باستقبال الاتصالات وإرسال البيانات في الوقت نفسه دون توقف البرنامج.

◀ بناء تطبيق عملي بلغة Python

- يُعد المشروع تدريباً عملياً على استخدام لغة Python في تطوير تطبيقات الشبكات.

3. آلية عمل المشروع

◀ عند تشغيل البرنامج يطلب من المستخدم ما يلي:

- إدخال اسم المستخدم.
- اختيار رقم المنفذ.

◀ بعد ذلك يقوم التطبيق بما يلي:

- إنشاء خادم محلي على الجهاز.
- فتح منفذ للاستماع للاتصالات.
- البحث عن الأجهزة الأخرى داخل الشبكة.
- الاتصال بالأجهزة المكتشفة.
- تبادل الملفات والرسائل والملاحظات.
- كل جهاز متصل يحتفظ بقائمة الأجهزة الأخرى المتصلة به.

4. المكونات الأساسية للمشروع

1.4. واجهة المستخدم النصية

يحتوي البرنامج على قائمة خيارات تظهر للمستخدم داخل الطرفية Terminal منها:

- اكتشاف الطلاب
 - عرض الأجهزة المتصلة
 - مشاركة ملف
 - طلب ملف
 - إنشاء ملاحظة
 - مشاركة ملاحظة
 - إرسال رسالة
 - بث رسالة جماعية
- وهذه الواجهة تسهل التعامل مع النظام.

2.4. الخادم المحلي Server

يقوم كل جهاز بإنشاء Server خاص به باستخدام مكتبة Socket في Python. وظيفته:

- استقبال الاتصالات الواردة.
- استقبال الملفات والرسائل.
- معالجة الطلبات.
- يعمل الخادم على منفذ محدد افتراضياً 5555

3.4. نظام اكتشاف الأجهزة Discovery System

يعتمد المشروع على تقنية Broadcast للبحث عن الأجهزة الأخرى داخل الشبكة. يقوم البرنامج بإرسال رسالة Broadcast إلى الشبكة، ثم تستجيب الأجهزة الأخرى وتتم عملية الاتصال. وهذه الطريقة تجعل اكتشاف المستخدمين تلقائياً دون الحاجة لإدخال العناوين يدوياً.

4.4. نظام مشاركة الملفات

يمكن للمستخدم اختيار ملف ومشاركته مع الآخرين.
آلية العمل:

- نسخ الملف إلى مجلد المشاركة.
- إرسال قائمة الملفات المتوفرة.
- عند طلب الملف يتم نقله عبر TCP.
- ويتم حفظ الملفات المستلمة داخل مجلد خاص.

5.4. نظام الملاحظات

- يسمح بإنشاء ملاحظات نصية ومشاركتها.
- تُحفظ الملاحظات داخل مجلد notes
- ثم تُرسل إلى بقية المستخدمين عند الطلب.

6.4. نظام الرسائل الفورية Chat

يقوم بإرسال:

- رسائل خاصة.
 - رسائل جماعية Broadcast.
- مما يجعل التطبيق شبيهاً بأنظمة المحادثة البسيطة.

7.4. إدارة الاتصالات

يحتفظ البرنامج بقائمة للأجهزة المتصلة self.peers والتي تتضمن:

- عنوان IP
- معلومات الاتصال
- وقت الاتصال

5. التقنيات المستخدمة

- لغة البرمجة Python 3
- المكتبات المستخدمة socket
- Threading لتشغيل عدة عمليات بالتوازي.
- Json لتحويل الرسائل إلى صيغة قابلة للإرسال.

- Os للتعامل مع الملفات والمجلدات.
- Hashlib يمكن استخدامها لاحقاً للتحقق من سلامة الملفات.
- Colorama لإظهار ألوان داخل الطرفية وتحسين شكل الواجهة.

6. أنواع الرسائل داخل النظام

يعتمد المشروع على أنواع متعددة من الرسائل، منها:

- hello التعريف بالمستخدم
- file_list عرض الملفات
- file_request طلب ملف
- file_data إرسال ملف
- note_share مشاركة ملاحظة
- note_list عرض الملاحظات
- chat_message إرسال رسالة

7. نقاط القوة في المشروع

- لا يحتاج خادم مركزي
- مما يقلل التكلفة ويزيد المرونة.
- سهولة الاستخدام
- واجهة بسيطة وواضحة.
- سرعة النقل داخل الشبكة المحلية لأن الأجهزة تتصل مباشرة ببعضها.
- قابلية التطوير

ويمكن إضافة:

- تشفير الملفات.
- تسجيل الدخول.
- واجهة رسومية GUI.
- نقل ملفات كبيرة.
- دعم الإنترنت الخارجي.

8. التحديات والمشكلات

- الأمان: حالياً لا يوجد تشفير للبيانات، مما قد يسمح بالتجسس على الاتصالات.
- إدارة الملفات الكبيرة: قد تواجه الشبكة ببطئاً عند نقل ملفات ضخمة.
- الاعتماد على الشبكة المحلية
- الاكتشاف التلقائي يعمل غالباً ضمن نفس الشبكة فقط.
- مشاكل المنافذ والجدران النارية: قد تمنع بعض إعدادات النظام الاتصالات.

9. تطورات مستقبلية مقترحة

يمكن تطوير المشروع بإضافة:

- تشفير AES للاتصالات.
- واجهة رسومية باستخدام Tkinter أو PyQt.
- قاعدة بيانات للمستخدمين.
- نظام صلاحيات.
- مشاركة عبر الإنترنت.
- ضغط الملفات قبل الإرسال.
- نظام تحقق من سلامة الملفات باستخدام SHA256.

10. الخاتمة

يعتبر مشروع نظام مشاركة الملفات والملاحظات باستخدام تقنية P2P مشروعاً عملياً مهماً لفهم أساسيات الشبكات والبرمجة الشبكية باستخدام Python. وقد نجح المشروع في تحقيق عدة وظائف مهمة مثل:

- الاتصال المباشر بين الأجهزة.
- مشاركة الملفات.
- تبادل الملاحظات.
- إرسال الرسائل.
- كما ساعد في تطبيق مفاهيم حقيقية في TCP/IP

- البرمجة متعددة الخيوط
- إدارة الاتصالات
- أنظمة الند للند

ويُعد هذا المشروع أساساً جيداً لتطوير أنظمة أكثر تقدماً في المستقبل، سواء في مجال مشاركة الملفات أو تطبيقات التواصل الشبكي.